EECE7352 Computer Architecture

Homework 2

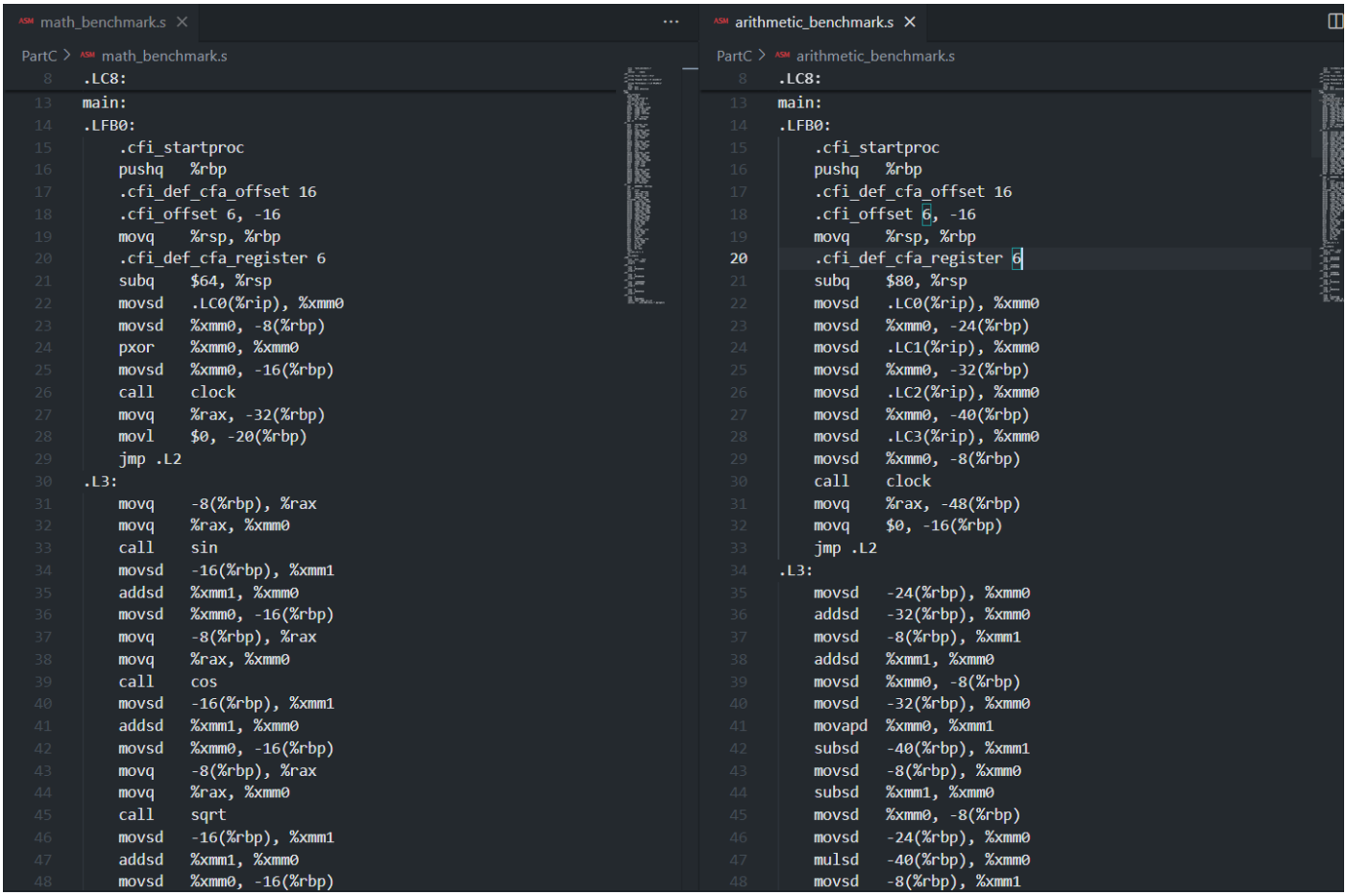
Sakthivel P. Sivakumar

NUID: 002312294

Part C:

For this part of the assignment, I wrote 2 benchmark program

1. **Arithmetic_Benchmark** – shows low-level floating point arithmetic instructions operating directly on XMM registers and memory
2. **Math_Bechmark –** uses floating point math functions with inputs/outputs , performing transcendental operations



math_benchmark.s

```
PartC > math_benchmark.s
 8  .LC8:
13  main:
14  .LFB0:
15      .cfi_startproc
16      pushq   %rbp
17      .cfi_def_cfa_offset 16
18      .cfi_offset 6, -16
19      movq    %rsp, %rbp
20      .cfi_def_cfa_register 6
21      subq    $64, %rsp
22      movsd   .LC0(%rip), %xmm0
23      movsd   %xmm0, -8(%rbp)
24      pxor    %xmm0, %xmm0
25      movsd   %xmm0, -16(%rbp)
26      call    clock
27      movq    %rax, -32(%rbp)
28      movl    $0, -20(%rbp)
29      jmp .L2
30  .L3:
31      movq    -8(%rbp), %rax
32      movq    %rax, %xmm0
33      call    sin
34      movsd   -16(%rbp), %xmm1
35      addsd   %xmm1, %xmm0
36      movsd   %xmm0, -16(%rbp)
37      movq    -8(%rbp), %rax
38      movq    %rax, %xmm0
39      call    cos
40      movsd   -16(%rbp), %xmm1
41      addsd   %xmm1, %xmm0
42      movsd   %xmm0, -16(%rbp)
43      movq    -8(%rbp), %rax
44      movq    %rax, %xmm0
45      call    sqrt
46      movsd   -16(%rbp), %xmm1
47      addsd   %xmm1, %xmm0
48      movsd   %xmm0, -16(%rbp)
```

arithmetic_benchmark.s

```
PartC > arithmetic_benchmark.s
 8  .LC8:
13  main:
14  .LFB0:
15      .cfi_startproc
16      pushq   %rbp
17      .cfi_def_cfa_offset 16
18      .cfi_offset 6, -16
19      movq    %rsp, %rbp
20      .cfi_def_cfa_register 6
21      subq    $80, %rsp
22      movsd   .LC0(%rip), %xmm0
23      movsd   %xmm0, -24(%rbp)
24      movsd   .LC1(%rip), %xmm0
25      movsd   %xmm0, -32(%rbp)
26      movsd   .LC2(%rip), %xmm0
27      movsd   %xmm0, -40(%rbp)
28      movsd   .LC3(%rip), %xmm0
29      movsd   %xmm0, -8(%rbp)
30      call    clock
31      movq    %rax, -48(%rbp)
32      movq    $0, -16(%rbp)
33      jmp .L2
34  .L3:
35      movsd   -24(%rbp), %xmm0
36      addsd   -32(%rbp), %xmm0
37      movsd   -8(%rbp), %xmm1
38      addsd   %xmm1, %xmm0
39      movsd   %xmm0, -8(%rbp)
40      movsd   -32(%rbp), %xmm0
41      movapd  %xmm0, %xmm1
42      subsd   -40(%rbp), %xmm1
43      movsd   -8(%rbp), %xmm0
44      subsd   %xmm1, %xmm0
45      movsd   %xmm0, -8(%rbp)
46      movsd   -24(%rbp), %xmm0
47      mulsd   -40(%rbp), %xmm0
48      movsd   -8(%rbp), %xmm1
```

Arithmetic Benchmark:

1. **addsd %xmm1, %xmm0**
   - **operand** used are two double-precision floats in XMM registers (%xmm1 and %xmm0)
   - **operation** is to add two values (scalar double add) and stores the result back in %xmm0

2. **subsd -40(%rbp), %xmm1**
    - operand used are a double from memory at stack offset -40(%rbp) and the value in %xmm1.
    - Operation is to subtract the memory operand from %xmm1 (scalar double subtract).

3. **mulsd %xmm1, %xmm0**
    - operands used are two doubles in XMM registers
    - Operation is to Multiply the operands (scalar double multiply), and store the result in %xmm0.

4. **divsd %xmm1, %xmm0**
    - **operands** are used to divide %xmm0 by %xmm1.
    - Operation is to divide the operands (Scalar double divide), and results are stored in %xmm0.

Math Benchmark:

1. **call sin**
    - Operands takes in the Input angle passed in %xmm0 as a double.
    - Operation is to computes the sine of the value in %xmm0 and returns result in %xmm0.

2. **call cos**
    - Operands takes Input angle in %xmm0.
    - Operation is to calculate cosine and place the result in %xmm0.

3. **call sqrt**
    - Operand takes Input value in %xmm0.
    - Operation is to calculate the square root of the input double, and store result in %xmm0.

4. **call log**
    - Operand takes Input value in %xmm0.
    - Operation is to compute natural logarithm of the input double, and store results in %xmm0.